

MM – B1

A Decimal Business RPN Calculator

exact base-10 money · TVM · cash flows · bonds · dates

User Manual

Version 1.1 · June 17, 2026

In the HP-12C lineage; companion to MM-11 and MM-14.
All arithmetic via the vendored IBM decNumber reference library.
Released under the GNU General Public License, version 3.

Contents

1	Introduction	2
1.1	Conventions used in this manual	2
2	Installing and building	2
3	The stack and RPN basics	2
4	Time value of money	3
4.1	Example: a mortgage	3
4.2	Example: saving toward a goal	3
4.3	Example: the true rate on a loan	4
5	Cash-flow analysis	4
6	Dated cash flows: XNPV and XIRR	4
7	Interest-rate conversions	4
8	Depreciation	5
9	Bonds	5
9.1	Periodic pricing	5
9.2	Dated pricing with accrued interest	5
10	Dates and day counts	5
11	Numerical methods and conventions	6
12	Word reference	6
13	License	7

1 Introduction

MM-B1 is a reverse-Polish (RPN) calculator for financial work. It belongs to the same family as the author's earlier MM-11 and MM-14 calculators — the same tokenizer, stack, registers, and word-dispatch loop on GNU readline — but it differs in two ways that matter for money.

First, **the arithmetic is decimal**. Binary floating point cannot represent a value as ordinary as 0.10, so a long chain of binary operations drifts away from the ledger in the least significant places. MM-B1 computes in base 10 with explicit round-half-even (banker's) rounding, so its answers agree with SQL `NUMERIC`, COBOL packed decimal, Java `BigDecimal`, and Python's `decimal` module — all implementations of the same General Decimal Arithmetic Specification. The engine is IBM's `decNumber`, the C reference implementation of that specification, vendored with the source so there is no external decimal dependency.

```
mmb> 0.1 0.2 +
x | 0.30
```

Second, there is a **real calendar** underneath the bond and cash-flow functions: actual day counts, the common day-count conventions, accrued interest, and settlement dates, so a bond can be priced on any business day rather than only on a coupon date.

1.1 Conventions used in this manual

Calculator sessions are shown in panels. What you type follows the `mmb>` prompt; the lines below it are what the calculator prints. The live display labels the stack levels `x`, `y`, `z`, `t` from the bottom up, with `x` the most recent value. In running text, words are shown like `?pmt`.

2 Installing and building

The only external dependency is GNU readline; the decimal library ships with the source.

```
make          build ./mmb1
make test     run the regression script tests.mmb
make run      build and start an interactive session
```

On Linux install readline first (`sudo apt install libreadline-dev`). On macOS readline is keg-only under Homebrew, so point the build at it:

```
brew install readline
make READLINE_PREFIX="$(brew --prefix readline)"
```

The decimal library is portable C and compiles identically on arm64 and x86_64.

3 The stack and RPN basics

In RPN you enter operands first and the operation last: `3 4 +` leaves 7. There is no equals key and there are no parentheses; the stack holds intermediate results. After every line MM-B1 prints the whole stack, newest at the bottom — you never need a print command interactively.

```
mmb> 3 4 +
x | 7.00
```

```
mmb> 10 *
  x |          70.00
mmb> 1234567.5 dup +
  y |          70.00
  x |    2,469,135.00
```

The bottom four levels are named **x y z t**; deeper levels are numbered. Use **dup**, **drop**, **swap**, and **over** to rearrange the stack, **clst** to clear it, and **depth** to push the current count. **N fix** sets the number of displayed decimal places; the stored value always keeps full 34-digit precision. Arithmetic words include **+** **-** ***** **/**, power **^**, **chs** (negate), **inv** (reciprocal), **sqrt**, **ln**, **exp**, and **abs**.

A line beginning text after **#** is a comment. When input is piped from a file the stack is *not* auto-printed — use **.** to print and pop the top value, or **.s** to show the stack — so scripts stay clean and predictable.

4 Time value of money

MM-B1 uses a *periodic* TVM model. The rate **i** is the interest rate **per period, in percent**, and **n** is the **number of periods**. A loan at 6% per year paid monthly is entered as **0.5 i** with **n** counted in months. There is no separate “payments per year” setting to reconcile.

The five registers satisfy

$$PV(1+r)^n + PMT(1+r\tau) \frac{(1+r)^n - 1}{r} + FV = 0, \quad r = \frac{i}{100},$$

where $\tau = 1$ in **begin** mode (annuity due) and 0 in **end** mode (the default). The **sign convention** is that cash received is positive and cash paid is negative. A solved payment that comes back negative is telling you it is an outflow.

Set any four of **n i pv pmt fv** (each consumes the top of the stack) and solve for the fifth with **?n**, **?i**, **?pv**, **?pmt**, or **?fv**; the solver stores its answer back into the register and pushes it. **tvm** prints the register state, **clrtvm** resets it.

4.1 Example: a mortgage

A \$300,000 loan over 30 years at 6% annual interest, paid monthly.

```
clrtvm
360 n  0.5 i  300000 pv  0 fv  end
?pmt          ->  -1,798.65    monthly payment
```

amort then prints the full schedule — interest, principal, and remaining balance for each period — from the current registers.

4.2 Example: saving toward a goal

Depositing \$500 at the end of each month for 30 years at 6% annual:

```
clrtvm
360 n  0.5 i  0 pv  -500 pmt
?fz          ->  502,257.52    balance after 30 years
```

To reach \$1,000,000 in 25 years at 7% annual, depositing at the *start* of each month, switch to annuity due:

```
clrtvm
300 n 0.5833333333 i 0 pv 1000000 fv begin
?pmt      -> -1,227.30      required monthly deposit
```

4.3 Example: the true rate on a loan

Borrow \$10,000, repay \$500 a month for 24 months:

```
clrtvm
24 n 10000 pv -500 pmt 0 fv
?i      -> 1.5131      percent per month (about 18.16% APR)
```

5 Cash-flow analysis

For a stream of cash flows at regular periods, build the cash-flow bank and evaluate net present value or internal rate of return. `cf` appends a flow (the first one sits at period 0 and is not discounted); `cfn` sets a repeat count on the most recently entered flow; `cfs` lists the bank and `clrcf` empties it. `npv` takes a periodic rate in percent; `irr` takes nothing and returns the rate in percent.

```
clrcf
-50000 cf 15000 cf 5 cfn
10 npv      -> 6,861.80      net present value at 10%
irr         -> 15.2382      internal rate of return, % per period
```

6 Dated cash flows: XNPV and XIRR

When flows fall on actual calendar dates rather than even periods, use the dated bank. A date is the plain integer YYYYMMDD. Append `amount date` pairs with `xcf`; `xcfs` lists them and `clrxcf` clears them. `xnpv` takes an annual rate in percent and discounts each flow by actual/365 from the date of the first flow — the same convention as the spreadsheet XNPV and XIRR functions. `xirr` returns the annual rate that sets XNPV to zero.

```
clrxcf
-10000 20080101 xcf
2750 20080301 xcf 4250 20081030 xcf 3250 20090215 xcf 2750 20090401 xcf
9 xnpv      -> 2,086.65      value at 9% per year
xirr        -> 37.3363      money-weighted annual return, %
```

7 Interest-rate conversions

`nom>eff` converts a nominal annual rate compounded m times a year into the effective annual rate; `eff>nom` does the reverse. Both take the rate in percent and m as the number of compounding periods.

```
18 12 nom>eff -> 19.5618      effective annual rate of an 18% APR card
```

8 Depreciation

Three methods are provided, each returning the depreciation for the requested year. **sl** (straight line) is constant and needs only cost, salvage, and life. **soyd** (sum-of-years-digits) and **db** (declining balance) also take the year; **db** additionally takes a factor (2 for double-declining balance).

```
50000 5000 7 sl      -> 6,428.57    straight line (every year)
50000 5000 7 1 soyd  -> 11,250.00    sum-of-years-digits, year 1
50000 5000 7 1 2 db  -> 14,285.71    double-declining, year 1
```

9 Bonds

9.1 Periodic pricing

bprice and **byield** price a bond on a coupon date with no calendar: a periodic coupon and yield (in percent), the number of whole periods, and the redemption value per 100 face.

```
3 3.5 20 100 bprice  -> 92.89      coupon below yield, so a discount
92.89 3 20 100 byield -> 3.5003    yield recovered from the price
```

9.2 Dated pricing with accrued interest

xprice and **xyield** price a bond for settlement on any date. They take the settlement and maturity dates, the annual coupon and yield in percent, the redemption per 100, the coupon frequency (coupons per year), and the day-count basis. The basis follows the spreadsheet numbering: 0 is 30/360 (US), 1 is actual/actual, 2 is actual/360, 3 is actual/365, and 4 is 30E/360 (European). Coupon dates are generated backward from maturity with end-of-month handling, and the result is the clean price — accrued interest netted out. **xaccr** returns the accrued interest by itself.

```
# settle    maturity  cpn% yld% redeem freq basis
20260615 20360515 4.25 4.5 100 2 1 xprice  -> 98.0146    clean price
20260615 20360515 4.25 2 1          xaccr   -> 0.3580    accrued per 100
```

The dirty (invoice) price is the clean price plus accrued interest. **xyield** inverts **xprice**: give it a price in place of a yield and it returns the yield. These functions reproduce the documented spreadsheet PRICE and YIELD worked examples to the displayed precision.

10 Dates and day counts

A date is the integer YYYYMMDD; for example 20260315 is 15 March 2026. **ddays** returns the actual number of days between two dates, **date+** shifts a date by a number of days, and **dow** returns the day of week (0 = Sunday). **yearfrac** returns the fraction of a year between two dates under a chosen basis (the same 0–4 numbering as the bonds).

```
20070115 20070315 ddays      -> 59          actual days between
20260315 30 date+             -> 20260414      shift a date by days
20260315 dow                   -> 0            day of week (0=Sun..6=Sat)
20070101 20080101 3 yearfrac  -> 1.0000      year fraction, act/365
```

11 Numerical methods and conventions

- Working precision is 34 significant digits with round-half-even. **fix** changes only the display (0–16 places); stored values are never truncated.
- The rate solvers (**?i**, **irr**, **xirr**, **byield**, **xyield**) have no closed form. MM-B1 scans the rate axis for a sign change and then bisection in decimal to convergence. Where a cash-flow stream admits more than one internal rate of return, only the first bracket found is reported.
- **amort** assumes end-of-period payments and works in magnitudes. Round the payment (or solve **?pmt** at the display precision) if you want the final balance to close to exactly zero.
- **xnpv** and **xirr** fix the year length at 365 days, matching the spreadsheet convention; **yearfrac** basis 1 uses ISDA actual/actual. For dated bonds, basis 0 (30/360) and basis 1 (actual/actual) are the best-tested paths.

12 Word reference

Group	Words
Arithmetic	<code>+ - * / ^ chs inv sqrt ln exp abs</code>
Business percent	<code>pct (x% of y), %chg, markup, margin</code>
Stack	<code>dup drop swap over depth clst . .s</code>
Constants	<code>pi e</code>
TVM set	<code>n i pv pmt fv</code>
TVM solve	<code>?n ?i ?pv ?pmt ?fv</code>
TVM mode	<code>begin end clrtvm tvn</code>
Cash flows	<code>cf cfn clrcf cfs npv irr</code>
Amortization	<code>amort</code>
Rate conversion	<code>nom>eff eff>nom</code>
Depreciation	<code>sl soyd db</code>
Bonds (periodic)	<code>bprice byield</code>
Dates	<code>ddays date+ dow yearfrac</code>
Dated cash flows	<code>xcf clrxcf xcfs xnpv xirr</code>
Dated bonds	<code>xprice xyield xaccr</code>
Storage	<code>s0..s9 (store, leaves stack), r0..r9 (recall)</code>
Display / meta	<code>fix help words quit</code>

The argument order is always postfix. A few of the multi-argument words, in the order their operands are pushed:

<code>y x pct</code>	<code>x% of y</code>
<code>cost salv life sl</code>	<code>straight-line depreciation</code>
<code>cost salv life year factor db</code>	<code>declining-balance depreciation</code>
<code>nom% m nom>eff</code>	<code>nominal -> effective annual</code>

cpn% yld% n redeem bprice	periodic bond price
settle mat cpn% yld% redeem freq basis xprice	dated bond clean price
settle mat cpn% price redeem freq basis xyield	dated bond yield
settle mat cpn% freq basis xaccr	accrued interest per 100
amount date xcf	append a dated cash flow

13 License

MM-B1 is released under the GNU General Public License, version 3, consistent with MM-11 and MM-14. The vendored decNumber library is distributed under the permissive ICU license, which the GPL permits redistributing alongside.